



Reto: detección de objetos en tiempo real

Su misión: desarrollar una solución de visión artificial capaz de detectar, localizar y clasificar objetos físicos a partir de video en vivo. Su prototipo deberá funcionar con los objetos que el jurado coloque frente a la cámara durante la demostración.

1 Equipo y recursos necesarios

Cada equipo debe contar con al menos una cámara de celular o laptop. La organización no proporcionará cámaras.

El equipo deberá llevar la computadora con la que desarrollará y ejecutará su solución. Puede usar la cámara integrada de la laptop o conectar la cámara de un celular como fuente de video. Si elige un celular, deberá configurar y probar la conexión e incluir las instrucciones en el README.

Entorno sugerido: Python 3.10 a 3.12 y OpenCV; PyTorch, Ultralytics (YOLO), TensorFlow, MediaPipe u otra biblioteca de visión. Pueden elegir otro entorno si justifican su decisión. El sistema debe poder ejecutarse en CPU; la GPU es opcional.

2 Objetos que deben reconocer

Taza	Botella	Tijeras	Destornillador	Teléfono celular
Taza de cerámica	Botella de plástico	Tijeras de oficina	Plano o Phillips	Smartphone

Los objetos que no pertenezcan a estas cinco clases deberán clasificarse como **DESCONOCIDO** o ser ignorados, sin asignarles incorrectamente una clase oficial.

3 Reglas de desarrollo

Pueden utilizar modelos preentrenados, transfer learning, datos propios, datasets públicos y documentación en línea. Citen las fuentes externas, repositorios y modelos utilizados. Cada equipo deberá explicar sus decisiones técnicas ante el jurado y entregar un repositorio Git o una carpeta comprimida con instrucciones claras.



Construyan, prueben y documenten

A Software de detección en vivo

40 PUNTOS

REQUISITO	PTS.
Captura de video en vivo con fuente configurable: cámara de laptop o conexión del celular.	4
Detección y localización de objetos mediante cajas delimitadoras (bounding boxes).	10
Clase y puntaje de confianza visibles para cada detección.	8
Detección de varios objetos simultáneamente en una misma escena.	6
FPS visibles en pantalla y un mínimo de 15 FPS en CPU.	6
Umbral de confianza configurable, cierre limpio con la tecla q y manejo del error cuando no hay cámara o se pierde la conexión.	6

B Robustez y falsos positivos

30 PUNTOS

El jurado probará distintas posiciones, distancias y orientaciones; varios objetos a la vez; objetos parcialmente ocultos; cambios de iluminación; objetos ajenos a las clases oficiales y un fondo vacío.

REQUISITO	PTS.
Implementar al menos una técnica para reducir falsos positivos: persistencia en varios frames, filtro por área o relación de aspecto, umbrales por clase o rechazo como desconocido, entre otras.	8
Justificar la técnica elegida en el README.	4
Alcanzar al menos 85 % de aciertos en 20 escenas evaluadas en vivo por el jurado.	18

C Evaluación y documentación

30 PUNTOS

REQUISITO	PTS.
Capturar al menos 100 imágenes con la cámara del equipo: 20 por clase, con variaciones de iluminación, fondo y posición; incluir sus etiquetas.	8
Describir el modelo, el etiquetado, el aumento de datos y los hiperparámetros principales. Si usan un modelo preentrenado sin ajuste, indicarlo y justificarlo.	8
Reportar precisión, recall, mAP y matriz de confusión sobre un conjunto de prueba separado, con un script reproducible.	8
README con instalación, ejecución, limitaciones conocidas y mejoras propuestas.	6



Lo que deben entregar

Entreguen un repositorio Git o una carpeta comprimida por equipo, con los siguientes elementos:

- 1 Código fuente.** main.py y los módulos necesarios para ejecutar la detección en vivo.
- 2 Dependencias.** requirements.txt con las versiones de las bibliotecas utilizadas, o su equivalente si eligen otro entorno.
- 3 README.md.** Instrucciones de instalación, configuración de la cámara, ejecución y evaluación; decisiones técnicas, fuentes externas, limitaciones y mejoras propuestas.
- 4 respuestas.md.** Explicación del modelo elegido, uso de modelos preentrenados o entrenamiento propio, técnica contra falsos positivos y decisiones principales del equipo.
- 5 Carpeta dataset/.** Al menos 100 imágenes capturadas con la cámara del equipo y sus etiquetas: 20 por clase. Identifiquen el conjunto de prueba separado del utilizado para entrenar o ajustar la solución.
- 6 Modelo.** Archivo del modelo utilizado o instrucciones precisas para descargarlo y cargarlo; si fue entrenado o ajustado por el equipo, incluyan los pesos resultantes.
- 7 Reporte de métricas.** metrics.md o un archivo .csv con precision, recall, mAP y matriz de confusión; incluyan el script y los pasos para reproducir la evaluación.
- 8 Video de demostración.** Máximo 2 minutos, mostrando el sistema funcionando con la cámara del equipo y ejemplos de detección en vivo.

Demostración ante el jurado

Presenten el prototipo funcionando con su propia cámara. El jurado colocará objetos frente a ella y probará las condiciones descritas en el reto. El equipo deberá explicar la solución y sus decisiones técnicas.

Puntuación y funciones adicionales

COMPONENTE	PUNTOS
A. Software de detección en vivo	40
B. Robustez y falsos positivos	30
C. Evaluación y documentación	30
TOTAL BASE	100

BONUS - hasta 10 puntos

Contador de objetos por clase en pantalla (+4);
seguimiento con ID por objeto entre frames (+3);
exportación a ONNX u optimización para un dispositivo embebido (+3).

PENALIZACIONES

-10 puntos si el código no se ejecuta siguiendo el README; -10 por código copiado sin citar; -5 por cada hora de retraso respecto al plazo de entrega que indique la organización.